

JAVASCRIPT (ES6+) – WHILE PETLJA

1. Uvod: Razlika između `for` i `while` petlje

U programiranju često koristimo petlje kako bismo ponavljali određeni blok koda. U JavaScriptu najčešće koristimo `for` i `while` petlju.

Ključna razlika:

- **`for` petlja** koristi se kada unaprijed znamo **točan broj ponavljanja**.
- **`while` petlja** koristi se kada ne znamo broj ponavljanja, ali znamo **uvjet koji mora biti zadovoljen** da bi se petlja nastavila izvršavati.

Važno je naglasiti:

`while` petlja provjerava uvjet **na početku svake iteracije**.

Ako je uvjet odmah na početku `false`, kod unutar petlje se **neće izvršiti niti jednom**.

Osnovna struktura i mjerenje vremena izvođenja

```
let uvjet = true;
let brojac = 0;

console.time('Početak')

while (uvjet) {
  if (brojac++ > 10000) {
    uvjet = false;
  }
  console.log(brojac);
}

console.timeEnd('Početak')
```

Objašnjenje:

Ovaj primjer prikazuje osnovnu strukturu `while` petlje i kontrolu njezina prekida.

Korak po korak:

1. Varijabla `uvjet` postavljena je na `true`, što znači da će petlja započeti s radom.

2. Varijabla `brojac` postavljena je na 0.
3. `console.time()` i `console.timeEnd()` mjere koliko je vremena potrebno da se petlja izvrši.
4. Petlja se izvršava sve dok je uvjet jednak `true`.
5. Izraz `brojac++` povećava vrijednost brojača za 1 pri svakoj iteraciji.
6. Kada `brojac` postane veći od 10000, uvjet se postavlja na `false`.
7. Time se prekida petlja.

Pedagoška napomena:

Ovaj primjer je odličan za objašnjenje:

- kako se ručno upravlja uvjetom izlaska iz petlje
- kako može nastati beskonačna petlja ako uvjet nikada ne postane `false`
- kako se mjeri vrijeme izvršavanja koda

Klasična `while` petlja – Simulacija baterije

```
let baterija = 30;

while (baterija > 0) {
  console.log(`Uređaj radi... Baterija na: ${baterija}%`);
  baterija -= 10;

  if (baterija === 0) {
    console.log("⚠ Uređaj se gasi: Baterija prazna!");
  }
}
```

Objašnjenje:

Ovaj primjer simulira pražnjenje baterije uređaja.

Logika rada:

1. Baterija počinje s 30%.
2. Petlja se izvršava dok je `baterija > 0`.
3. U svakom prolazu:
 - Ispisuje se trenutna razina baterije.
 - Baterija se smanjuje za 10%.
4. Kada baterija dosegne 0%, ispisuje se poruka upozorenja.

Što polaznici ovdje uče:

- `while` se koristi kada trajanje ovisi o stanju sustava.
- Uvjet se provjerava prije svake iteracije.
- Ako bi baterija bila inicijalno 0, petlja se ne bi izvršila niti jednom.

Obrada niza – Pražnjenje reda čekanja

```
const klijenti = ['Ana', 'Marko', 'Ivan', 'Lucija'];

while (klijenti.length > 0) {
  let trenutni = klijenti.shift();
  console.log(`Uslužujemo klijenta: ${trenutni}. Preostalo u redu:
  ${klijenti.length}`);
}
```

Objašnjenje:

Ovdje koristimo `while` za rad s nizom čija se duljina mijenja tijekom izvođenja.

Ključni elementi:

- `klijenti.length > 0` → petlja traje dok niz nije prazan.
- `.shift()` metoda:
 - uklanja prvi element iz niza
 - vraća uklonjeni element
 - smanjuje duljinu niza

Zašto je `while` ovdje idealan?

Ne znamo točan broj ponavljanja unaprijed – znamo samo da želimo raditi dok red nije prazan.

Pedagoška vrijednost:

Ovaj primjer pokazuje:

- dinamičku promjenu podataka tijekom rada petlje
 - kako se stanje sustava mijenja nakon svake iteracije
 - praktičnu primjenu u simulaciji realnih sustava (red čekanja)
-

Generiranje do nasumičnog rezultata

```
let kocka = 0;
let brojBacanja = 0;

while (kocka !== 6) {
  kocka = Math.floor(Math.random() * 6) + 1;
  console.log(`Bacanje br. ${++brojBacanja}: Dobili smo broj ${kocka}`);
}
```

Objašnjenje:

Ovo je klasičan primjer situacije u kojoj ne znamo koliko će se puta petlja izvršiti.

Logika:

1. Varijabla `kocka` počinje s 0.
2. Petlja traje dok ne dobijemo broj 6.
3. Svako bacanje:
 - o generira broj od 1 do 6
 - o povećava brojač bacanja
4. Kada dobijemo 6, petlja se prekida.

Što polaznici ovdje razumiju:

- `while` je idealan kada broj ponavljanja ovisi o slučajnosti.
- Uvjet se provjerava prije svake iteracije.
- Ovo je tipičan primjer simulacije realnog procesa.

`continue` i rizik od beskonačne petlje

```
let i = 0;

while (i < 5) {
  i++;

  if (i === 3) {
    console.log(`[X] Preskačemo broj ${i}`);
    continue;
  }

  console.log(`Obrađujem stavku: ${i}`);
}
```

Objašnjenje:

Ovdje se koristi naredba `continue`.

Kako `continue` radi?

- Prekida trenutnu iteraciju.
- Vraća izvršavanje na početak petlje.
- Ponovno se provjerava uvjet.

Kod `while` petlje inkrement (npr. `i++`) mora biti izveden prije `continue`.

Ako bismo napisali:

```
if (i === 3) {  
    continue;  
}  
i++;
```

petlja bi postala beskonačna jer se `i` nikada ne bi povećavao kada je 3.

Beskonačna petlja i `break`

```
let brojacSlanja = 0;  
  
while (true) {  
    brojacSlanja++;  
    console.log(`Šaljem paket podataka br. ${brojacSlanja}...`);  
  
    if (brojacSlanja === 4) {  
        console.log("🛑 Sigurnosni prekid: Dosegnut limit slanja!");  
        break;  
    }  
}
```

Objašnjenje:

Ovdje koristimo namjernu beskonačnu petlju.

`while (true)` znači da je uvjet uvijek zadovoljen.

Petlja će se izvršavati beskonačno osim ako:

- ne koristimo `break`
- ne dođe do greške
- ne prekinemo program